

Counter Allocation Methodology: Holding Check-in Wait Below Target

Author: Yash Moitra **Institution:** Delhi International Airport Limited **Date:** June 30, 2026

This document records, step by step, the wait-time logic and the counter-allocation method used to keep each departing passenger's check-in wait below a target (20 minutes, and separately 40 minutes). It follows the project Ground Rules: variables and terms are consistent with `Variable_Names.md` and `Definitions.md`, exhibits are captioned and numbered (continuing the canonical Equation sequence from `Methodology.md`, which ends at Equation (6)), authorship and dating follow Rule 4, and no em dashes are used (Rule 6).

I. Objective

Given the entry times of every passenger (one value per passenger per entry-time model) and a check-in service speed, determine the number of check-in counters, possibly varying through the day, that keeps the queueing wait of every passenger below a target. Two targets are solved: 20 minutes (the OMDA economy benchmark, Table 5) and 40 minutes. The required counters are then compared against the physical supply of 168 counters (Section IX).

II. Inputs

1. `seats.xlsx` (cached to `seats_ready.pkl` for speed): 39,610 passengers, each carrying an entry time under 104 entry-time models (families Normal N1 to N13, Uniform U1, ExpGrowth EG0 to EG5, ExpDecay ED0 to ED5, SkewLeft, SkewRight). Entry is stored as minutes before departure at load factor 1.
2. `asimplified.csv`: per-flight Scheduled departure (SOBT), Airline code, and Ground Handler (GHA). Joined to passengers on Flight Designator. GHA coverage is 100 percent; the schedule carries AISATS, BFS, Ramp 360, and AIESL.
3. Counter inventory by island (Section IX), `counters_by_row.csv`.

III. Time convention

Equation (7), clock entry. A passenger's entry instant on the clock is $a = \text{dep} - e$, where dep is the departure time in minutes after midnight and e is the stored entry time in minutes before departure. Queues are formed and ordered by a . A negative a denotes the evening before (a 00:05 departure with $e = 165$ enters at $a = -160$, that is 21:20 the previous day).

IV. Single-desk FIFO wait (the core logic)

For one counter serving a queue first-in-first-out at a constant service time t minutes per passenger, order the queue by arrival $a_1 \leq a_2 \leq \dots$. Then:

- **Equation (8), service start.** $s_i = \max(a_i, d_{\{i-1\}})$.
- **Equation (9), departure / processed time.** $d_i = s_i + t$. Equivalently the processed time equals $\text{entry} + \text{wait} + t$, which is the quantity placed next to each entry cell.
- **Equation (10), wait.** $w_i = s_i - a_i = \max(0, d_{\{i-1\}} - a_i)$.

Three equivalent forms are used and were cross-checked against Equation (10) on 20,000 random sequences with zero discrepancy:

- **Equation (11), increment (Lindley).** $w_i = \max(0, w_{i-1} + t - (a_i - a_{i-1}))$.
- **Equation (12), closed form.** $w_i = \max \text{ over } j \text{ in } [0, i-1] \text{ of } (j*t - (a_i - a_{i-j}))$.
- **Equation (13), elapsed form (expands Equation (5)).** $w_i = L_i*t - e_i$, where L_i is the number of passengers still ahead in the system when i arrives and e_i is the service time already elapsed on the passenger then in service. This is the “percentage of t already elapsed” view: if a fraction ϕ of the in-service passenger’s t is done, the residual is $(1 - \phi)*t$ and $w_i = (L_i - \phi)*t$.

Correction to the hand worksheet. For arrivals at minutes 0, 1, 2 with t between 60 and 120 seconds, the third passenger’s wait is $2t - 2$, not $(t-1) + (t-2) = 2t - 3$. The second stacked term must be measured from the predecessor’s completion, that is $t - (\text{gap between arrivals}) = t - 1$, giving $(t-1) + (t-1) = 2t - 2$. Verified: at $t = 1.5$ min the wait is 1.0 min, not 0.0. The general rule is the cumulative recursion of Equation (8), not a subtraction of each absolute arrival time from a single $(0 + t)$.

V. Multi-counter extension

Assumption 5 (single pooled queue per group). Within a group (Section VII) all passengers form one queue feeding c identical counters, FIFO, work-conserving.

Equation (14), c-counter recursion. With c identical counters and constant t , passenger i reuses the counter that served passenger $i - c$, so $d_i = \max(a_i, d_{i-c}) + t$. The system therefore decomposes exactly into c interleaved single-counter queues (passengers $i, i+c, i+2c, \dots$ by sorted arrival). This lets the verified single-counter engine compute the c -counter waits, and was validated against a direct c -server simulation.

VI. Service regimes

Two service speeds are solved in parallel:

- **r200:** 18 passengers per counter per hour, that is $t = 200$ seconds = 10/3 minutes.
- **r180:** 180 seconds per passenger, that is $t = 3.0$ minutes (20 per hour).

VII. Groupings (who shares counters)

Counters are pooled at three granularities, solved independently:

1. **Per flight** (164 groups): each flight’s passengers share their own counters.
2. **Per airline** (57 groups): all of an airline’s passengers share counters.
3. **Per GHA** (4 groups): AISATS, BFS, Ramp 360, AIESL.

VIII. Allocation rule

Equation (15), counter capacity. A single counter clears $N = \text{target} / t$ passengers within the target window. For $\text{target} = 20$: $N = 6.0$ (r200) and 6.667 (r180). For $\text{target} = 40$: $N = 12.0$ (r200) and 13.333 (r180). This N is the “passengers per counter” bottleneck the worksheet referred to.

Equation (16), counters at a minute (the loop). For each group and each minute m , let $A_W(m)$ be the number of arrivals in the trailing window $(m - \text{target}, m]$. The counters required

are $D(m) = \text{ceil}(A_W(m) / N)$. This is the “if passengers per counter would exceed N , add a counter” rule applied continuously, giving a time-varying staffing profile $D(m)$. The group peak is $\max_m D(m)$; the airport peak is $\max_m$ of the sum of $D(m)$ across groups (counters needed at the same instant).

Validation: staffing each flight at its D profile yields realized maximum waits well inside the target (for N1 at the 20-minute target, group maxima average about 2 to 4 minutes), confirming the rule is feasible. It is deliberately conservative, see Section XI.

IX. Supply: counter inventory

The physical check-in counters by island (the airline assignments noted on the source are deliberately ignored here):

A 14, B 8, C 14, D 12, E 14, F 12, G 14, H 14, J 10, K 14, L 14, M 14, N 14.

Total supply = 168 counters across 13 islands (no island I; maximum 14 per island). Stored in `counters_by_row.csv`.

X. Results (entry-time model N1, mean 150 min, SD 30)

Airport peak counters required (time-varying rule, Equation (16)) against the 168 supply:

Grouping	Target	r200 (200s)	r180 (180s)
Per flight	20 min	171 (over by 3)	159 (fits)
Per flight	40 min	171 (over by 3)	156 (fits)
Per airline	20 min	164 (fits)	150 (fits)
Per airline	40 min	161 (fits)	146 (fits)
Per GHA	20 min	159 (fits)	143 (fits)
Per GHA	40 min	154 (fits)	139 (fits)

Reading: at 180 seconds per passenger the 20-minute target is met within 168 counters under every grouping. At 200 seconds per passenger the per-flight pooling is marginally short (171 versus 168); pooling counters across airlines or GHAs absorbs the peak and fits. The 40-minute target is met everywhere except per-flight at 200 seconds.

Across all 104 entry-time shapes the per-flight 20-minute peak ranges from 146 to 253 (r200) and 136 to 229 (r180). The most back-loaded, last-minute shapes (for example the steep ExpGrowth EG5) push demand well above 168 and would breach the target, so passenger arrival behaviour, not only the counter count, governs feasibility.

XI. Notes and limitations

1. The Equation (16) rule is a sufficient, conservative allocation: it sizes each window so even a worst-case burst clears within the target, so realized waits sit far below the target. The true minimum peak is therefore at or below the figures in Section X. A tighter time-varying optimum can be solved next.
2. Holding a constant number of counters per group all day is materially less efficient, especially for pooled groupings where idle periods still hold counters open, and is not recommended.

3. One pooled FIFO queue per group is assumed (Assumption 5). Desk eligibility by class and loyalty (Equation (6), Tables 3 and 4) is not yet applied.
4. Load factor is 1 (every seat a passenger); lower load factors reduce demand roughly proportionally.
5. Service is homogeneous across counters and constant in time (Assumption 3); no setup, breaks, or per-airline speed differences.
6. Counters are integer and assumed reassignable instantly; island boundaries and the maximum of 14 counters per island are not yet imposed, and rows are not yet mapped to airlines or GHAs.
7. Times use the continuous clock of Equation (7); processed times after midnight are kept on the same axis.

XII. Files produced

`counters_by_row.csv` (supply); `allocation_summary.csv` and `.xlsx` (per grouping, regime, target, and all 104 models: airport peak, group peaks, total counter-minutes, feasibility against 168); `alloc_heuristic_{flight,airline,gha}.csv`; `profile_{flight,airline,gha}_N1.csv` (minute-by-minute desk profile for N1); `alloc_constant_staffing_N1.csv` (the less efficient constant scheme, for reference); plus the wait engine `wait_time.py`, the build scripts, and the worked example `worked_example.xlsx`.

XIII. Next steps

Solve the tighter time-varying minimum (binary search at the target rather than the conservative rule); impose island and 14-per-island constraints and assign islands to airlines or GHAs; apply desk eligibility by class and loyalty; sweep load factor.